MODULE 13 · SECTION 2 - INGESTING AND PROCESSING THE DATA

Windows: Fixed, Sliding, Session, Global

Event time against processing time, the four window types, and picking the right one from the question's wording.

SECTION 2 of 5 EXAM WEIGHT 25% MODULE 13 of 45

admin@sthithapragna.com

A stream never ends, so you can never total it. **Windows cut it into pieces you can finish.**

admin@sthithapragna.com

Event time or processing time?

EVENT TIME

When it happened

- The meter's own timestamp
- reading_ts at Larkspur
- Stable, however late it arrives
- What the business means

PROCESSING TIME

When the pipeline saw it

- The clock on the worker
- Changes with every delay
- Fine for system monitoring
- Wrong for billing and demand

FOR LARKSPUR

Meters can upload up to 72 hours late. Only event time puts a late reading in the right half hour.

admin@sthithapragna.com

Four ways to cut a stream

WINDOW	SHAPE	THINK OF
Fixed, or tumbling	Equal, back to back, no overlap	A bus timetable: every 30 minutes
Sliding, or hopping	A length and a period, overlapping	A moving average on a share ticker
Session	Per key, ends after a quiet gap	A shop visit that ends after 15 quiet minutes
Global	One window for everything	A running total since the very start

REMEMBER

Learn both names for fixed and sliding. Google's docs say tumbling and hopping; Beam says fixed and sliding.

admin@sthithapragna.com

Fixed: one reading, one window

- Equal size, **no overlap**. Every element lands in **exactly one** window.
- Larkspur's **half-hour settlement periods** are fixed windows of 30 minutes.
- Windows line up with the clock, in **UTC**. Use an offset for other alignments.
- Cheapest to run, because nothing is counted twice.

> THE SIGNAL
>
> 'Every hour', 'per half hour', 'each day' all point to fixed windows.

Sliding: the last X, every Y

- Two numbers: the **length** and the **period**, how often a new window starts.
- Windows **overlap**, so each element lands in several of them.
- A 60-minute window every 5 minutes puts each reading in **12 windows**.
- For smooth, rolling numbers: 'usage in the last hour', refreshed often.

> THE SIGNAL
>
> 'The last 30 seconds, every 5 seconds' is a sliding window: length 30, period 5.

admin@sthithapragna.com

Session: until things go quiet

- Defined by a **gap duration**. A pause longer than the gap ends the session.
- **Per key**: each engineer, customer or device gets its own sessions.
- Sessions **vary in length**, and merge when new events close the gap.
- Right for visits, journeys and bursts of activity.

> THE SIGNAL
>
> 'Until there is no activity for 15 minutes' is a session window with a 15-minute gap.

admin@sthithapragna.com

Global: the default, and its catch

- Every PCollection starts in **one global window**. Fine for bounded batch data.
- On a stream, a **GroupByKey** in the global window, with the default trigger, **fails at build time**.
- Fix it with a **non-global window**, or a **trigger** that fires repeatedly.
- Global plus a trigger gives 'since the start, updated every 10 minutes'.

> REMEMBER
>
> Inputs to a CoGroupByKey must share the same windowing, or the pipeline will not build.

Three windows on Larkspur's stream

```
stamped = parsed | "Event time" >> beam.Map(
    lambda r: window.TimestampedValue(r, r["reading_epoch"]))
by_region = stamped | "Key" >> beam.Map(lambda r: (r["region"], r["kwh"]))
half_hours = (by_region
    | "Fixed 30m" >> beam.WindowInto(window.FixedWindows(30 * 60))
    | "Sum" >> beam.CombinePerKey(sum))
last_hour = (by_region
    | "Slide 60m by 5m" >> beam.WindowInto(window.SlidingWindows(60 * 60, 5 * 60))
    | "Sum hour" >> beam.CombinePerKey(sum))
visits = engineer_events | "Gap 15m" >> beam.WindowInto(window.Sessions(15 * 60))
```

`2` Event time from the meter, not the worker's clock.

`5, 8` Half hours counted once; a rolling hour refreshed every 5 minutes.

THE POINT

Same stream, three questions. The window, not the data, decides which question is answered.

admin@sthithapragna.com

Grouping happens per key and per window

- A **GroupByKey** groups by key **inside each window**. One key, two windows, two results.
- Window first, then group. The window decides **when**, the key decides **what**.
- A key that is huge in every window is a **hot key**, every window.
- For per-region totals, the key is the region. For per-meter, the key is the meter.

> THE SIGNAL
>
> 'Per meter, per half hour' means key by meter, then fixed 30-minute windows.

admin@sthithapragna.com

Windows without Beam

- In batch SQL, a fixed window is just **GROUP BY** a truncated or bucketed timestamp.
- **Continuous queries** offer **TUMBLE** for fixed windows. Preview.
- TUMBLE windows use the row's **change timestamp**, up to 24 hours long.
- No sliding or session windows in continuous queries. Those need Dataflow.

> THE TIEBREAKER
>
> 'Sessions' or 'rolling windows' on a live stream still points to Dataflow.

admin@sthithapragna.com

Phrases that pick the window

THE QUESTION SAYS	WINDOW
Every half hour, per day, each hour	Fixed, or tumbling
The last 10 minutes, every minute	Sliding, or hopping
Until 15 minutes of inactivity	Session, per key
Since the start, updated every few minutes	Global, with a repeating trigger
When it happened, late uploads	Event time, not processing time
Per meter, per half hour	Key by meter, then fixed windows

THE PATTERN

Underline the time words in the question. They name the window.

admin@sthithapragna.com

> Westgrid wants Larkspur's total demand for each region, for each half-hour settlement period.
>
> Each reading must count towards exactly one period, the one in which it was taken.

Which windowing should the pipeline use?

A. Sliding windows of 30 minutes, starting every 5 minutes, by region

B. Fixed 30-minute windows on event time, keyed by region

C. Session windows with a 30-minute gap, keyed by region

D. The global window, with a trigger every 30 minutes of processing time

◆ PAUSE AND DECIDE

admin@sthithapragna.com

Which windowing should the pipeline use?

A Sliding windows

Each reading lands in six windows, not exactly one.

B Fixed, event time

One reading, one half hour: the one it was taken in.

C Session windows

Sessions follow activity. Regions never go quiet for 30 minutes.

D Global, processing time

Totals by arrival time, so late readings land in the wrong period.

THE SIGNAL

'Each half hour' plus 'exactly one period' plus 'when it was taken': fixed windows on event time.

admin@sthithapragna.com

> The customer app shows each household its electricity use over the last 60 minutes.
>
> Customers want the number refreshed every 5 minutes, not once an hour.

Which windowing fits this best?

A Fixed 60-minute windows, keyed by meter

B Session windows with a 5-minute gap, keyed by each household's meter

C Fixed 5-minute windows, summed in the app afterwards

D Sliding windows, 60 minutes long, starting every 5 minutes

◆ PAUSE AND DECIDE

admin@sthithapragna.com

Which windowing fits this best?

A Fixed 60-minute windows

Right total, but it updates only once an hour.

B Sessions, 5-minute gap

Meters report every half hour, so every reading is its own session.

C Fixed 5-minute, summed later

Moves the windowing into the app, and every client repeats it.

D Sliding 60 by 5

A rolling hour, recomputed every five minutes, exactly as asked.

THE SIGNAL

'The last 60 minutes, every 5 minutes' is a sliding window: length 60, period 5.

admin@sthithapragna.com

> Field engineers' tablets send diagnostic events while they work at a site. Larkspur wants one summary per site visit, per engineer.
>
> A visit is over when an engineer's tablet has sent nothing for 15 minutes. Visits last anything from 10 minutes to 6 hours.

Which windowing fits this best?

A. Session windows with a 15-minute gap, keyed by engineer

B. Fixed 15-minute windows, keyed by engineer

C. Sliding windows of 6 hours, starting every 15 minutes

D. Fixed 6-hour windows, with every visit trimmed to fit inside one

◆ PAUSE AND DECIDE

admin@sthithapragna.com

Which windowing fits this best?

A Sessions, 15-minute gap

Each engineer's visit ends after 15 quiet minutes, whatever its length.

B Fixed 15-minute windows

Cuts a two-hour visit into eight pieces.

C Sliding 6-hour windows

Overlapping windows, so each visit appears many times.

D Fixed 6-hour windows

Visits cross window edges, and get split or merged by the clock.

THE SIGNAL

'Ends after 15 quiet minutes' plus 'varies in length' plus 'per engineer' is a session window.

admin@sthithapragna.com

> Larkspur wants a running total of energy for each meter, counted from the day it was installed.
>
> The total should be refreshed every 10 minutes, and never reset.

How should the pipeline window the data?

A Fixed 10-minute windows, keyed by meter

B Sliding windows one year long, starting every 10 minutes per meter

C The global window per meter, with a trigger every 10 minutes

D Session windows per meter, with a 10-minute gap

◆ PAUSE AND DECIDE

admin@sthithapragna.com

How should the pipeline window the data?

A Fixed 10-minute windows

Each window holds only its own ten minutes, so the total resets.

B One-year sliding windows

Still forgets anything older than a year, and costs a fortune.

C Global, repeating trigger

One window that never closes, with a result every ten minutes.

D Session windows

A reading every half hour means a new session every time.

THE SIGNAL

'Since installation, never reset, updated every 10 minutes': the global window plus a repeating trigger.

admin@sthithapragna.com

> Larkspur's half-hourly demand totals keep disagreeing with the meters. Readings that arrive late are counted in whichever half hour they reach the pipeline.
>
> Some meters upload up to 72 hours late after losing signal.

What is the root cause, and the fix?

A Processing-time windows: window on the meter's event timestamp instead

B Too few workers: raise the maximum, so that readings are processed sooner

C Fixed windows: switch to sliding windows, to smooth out the totals

D A missing dead-letter output: send late readings to an error table

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the root cause, and the fix?

A Use event time

Each reading carries its own time, so late ones rejoin the right half hour.

B More workers

Workers cannot speed up a meter that had no signal for three days.

C Sliding windows

Smoother numbers, still in the wrong half hours.

D Dead-letter late readings

Late readings are valid. Throwing them aside loses real demand.

THE SIGNAL

'Counted when they reach the pipeline' is processing time. The fix is event time.

> A new streaming pipeline reads Pub/Sub and calls GroupByKey to count readings per meter. No windowing or trigger is set.
>
> The job fails before it starts, with an error about the GroupByKey.

What should the developer change?

A Raise the worker count, so the grouping has enough memory

B Apply a window, or a repeating trigger, before the GroupByKey

C Run the same pipeline as a batch job over the subscription

D Replace the GroupByKey with a side input holding the meter counts

◆ PAUSE AND DECIDE

What should the developer change?

A More workers

It failed at construction, before any worker ran.

B Window before grouping

An unbounded GroupByKey needs a window or a trigger, or Beam rejects it.

C Run as batch

A subscription is unbounded, and batch loses the streaming result.

D A side input

Side inputs are for lookups, not for counting the stream itself.

THE SIGNAL

Unbounded data plus GroupByKey plus no window or trigger fails at build time.

admin@sthithapragna.com

Module 13 in five lines

- Window on **event time**, the meter's clock, not the worker's.
- **Fixed** for 'every half hour'. **Sliding** for 'the last X, every Y'.
- **Session** for 'until quiet for N minutes', per key.
- **Global** plus a repeating trigger for 'since the start'.
- An unbounded GroupByKey needs **a window or a trigger**, or it will not build.

COMING UP

Next: module 14, watermarks, late data and triggers.

admin@sthithapragna.com